Reliable Federated TinyML Deployment for IoT Security (WIP)

Anonymous Author(s)

Abstract

The growing deployment of Internet of Things (IoT) devices has increased the need for privacy-preserving intrusion detection systems that operate directly on resource-constrained hardware. Federated Learning enables collaborative model training without sharing raw data, but conventional federated models are often too large and unstable for deployment on microcontroller-class devices. TinyML techniques enable compact neural networks but are typically designed for inference-only workloads.

This work investigates combining Federated Learning with TinyML-based model compression for intrusion detection in IoT environments. We evaluate compression strategies including knowledge distillation, structured pruning, and quantization within a federated training pipeline. Preliminary results show that training stability plays a critical role in federated TinyML systems. In particular, server-coordinated cosine learning-rate scheduling improves Attack Recall from 46.7% to 93.85% while enabling substantial model compression and efficient edge deployment. These findings provide insights for designing lightweight and privacy preserving intrusion detection systems for IoT devices.

CCS Concepts: • **Security and privacy → Intrusion detection systems**.

Keywords: Federated Learning, TinyML, Intrusion Detection Systems, IoT Security, Model Compression

1 Introduction

The rapid proliferation of Internet of Things (IoT) devices has enabled large-scale deployment of data-driven applications across domains such as smart homes, healthcare, and industrial systems [1]. These applications increasingly rely on machine learning models trained on sensitive data, raising concerns about privacy, data ownership, and communication overhead in centralized training.

Federated Learning (FL) addresses these challenges by enabling collaborative training without sharing raw data [8, 10, 12]. However, deploying FL in IoT environments remains difficult due to limited memory, computation, and energy resources, as well as the large model sizes typically assumed in standard FL pipelines [9].

TinyML enables neural networks to run directly on microcontrollers by optimizing model size, memory footprint, and inference latency [2]. However, most TinyML systems focus on inference-only workloads, and integrating TinyML with federated training remains an open challenge, particularly for security-critical applications such as intrusion detection.

Machine learning-based intrusion detection systems (IDS) can achieve strong detection performance [7], but deploying them on IoT devices requires lightweight models while maintaining high attack recall. In our early experiments on the CIC-IDS2017 dataset, a federated IDS model trained with a fixed learning rate achieved only 46.7% attack recall, despite high overall accuracy, due to unstable optimization under class imbalance. Additional challenges include deployment instability (e.g., BatchNorm interactions) and performance degradation under aggressive compression [4–6].

To address these challenges, we investigate combining FL with TinyML-oriented compression techniques for IoT intrusion detection. We evaluate knowledge distillation, structured pruning, quantization-aware training (QAT), and post-training quantization (PTQ), and analyze their impact on detection performance, training stability, and deployment feasibility. We further examine adversarial robustness using the Fast Gradient Sign Method (FGSM) [3].

Our results show that training stability is a key factor in federated TinyML systems. In particular, server-coordinated cosine learning-rate scheduling improves attack recall from 46.7% to 93.85%. In addition, the proposed compression pipeline significantly reduces model size and inference latency while preserving strong detection performance. These findings provide practical guidelines for designing lightweight and privacy-preserving intrusion detection systems for resource-constrained IoT devices.

2 Related Work

Federated Learning (FL) enables collaborative model training across distributed clients without sharing raw data [8, 12]. This paradigm is well suited for IoT environments with privacy and bandwidth constraints, but practical deployment remains challenging due to communication overhead, client heterogeneity, and the difficulty of running large models on resource-constrained devices [9].

TinyML addresses these limitations by enabling neural networks to run directly on microcontrollers through model size, memory, and latency optimization [2]. However, most TinyML systems focus on inference-only workloads, and integrating TinyML with federated training requires lightweight models and carefully designed pipelines.

LCTES '26, Boulder, Colorado, United States
2026. ACM ISBN 978-x-xxxx-xxxx-x/26/06
https://doi.org/10.1145/xxxxxxx.xxxxxxx

Model compression techniques are widely used to enable efficient deployment on constrained hardware. Structured pruning removes redundant parameters, knowledge distillation transfers knowledge from large teacher models to compact students, and quantization reduces numerical precision for efficient inference [4–6]. Quantization-aware training (QAT) and post-training quantization (PTQ) are commonly used to convert floating-point models into efficient integer representations.

Machine learning-based intrusion detection systems (IDS) have shown strong performance in detecting network attacks, but most approaches assume centralized training and resource-rich environments. In addition, robustness to adversarial attacks has been widely studied, with methods such as the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD) commonly used for evaluation [3].

Despite these advances, few studies have jointly addressed federated learning, TinyML deployment constraints, and intrusion detection in IoT systems. Achieving high detection performance while maintaining compact models and reliable embedded deployment remains an open challenge. This work addresses this gap by evaluating compression strategies and training techniques within a federated TinyML pipeline for resource-constrained IoT intrusion detection.

3 Approach

The goal of this work is to maintain high intrusion detection performance while minimizing model size to enable deployment on resource-constrained IoT devices, while preserving accuracy and F1-score.

3.1 Federated Training Baseline

We begin with a baseline Federated Learning (FL) framework in which edge devices locally train an intrusion detection model and periodically transmit model updates to a central server, which aggregates updates to redistributed to clients [8, 10, 12]. To address class imbalance in intrusion detection data, we employ Focal Loss

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t),$$

where p_t is the predicted probability for the ground-truth class. In our experiments we set $\alpha = 0.7$ and $\gamma = 2.0$. Training runs for 80 federated communication rounds and establishes the reference performance before compression. Early experiments revealed unstable convergence under a fixed learning rate due to dataset imbalance. To stabilize training we introduce server-coordinated cosine learning-rate decay [11], where the global learning rate η evolves across T rounds as

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(t\pi/T)).$$

3.2 Model Compression Experiments

To reduce model size while maintaining detection performance, we evaluate compression techniques including knowledge distillation and structured pruning [4, 5]. Multiple configurations are explored to identify the best trade-off between detection accuracy and model efficiency.

The final compression pipeline consists of BatchNorm folding, knowledge distillation, structured pruning, and optional quantization-aware fine-tuning before conversion to TFLite INT8 format. Each configuration is evaluated using model size, accuracy, and F1-score to determine which compression strategies best support deployment on resource-constrained IoT hardware.

3.3 Model Selection for Deployment

From the compression experiments we select four representative configurations illustrating different performance–efficiency trade-offs:

- Most Compressed: smallest model with acceptable performance
- Most Accurate: highest accuracy and F1-score
- Balanced-Compressed: moderate compression with strong performance
- Balanced-Accurate: high accuracy with moderate model size

These configurations allow practitioners to select models based on deployment constraints. Highly constrained devices may require the most compressed configuration, while more capable devices can adopt balanced or accuracy-oriented models.

4 Experimental Results

We report experimental results for federated TinyML intrusion detection, focusing on training stability, compression efficiency, quantization strategies, and deployment reliability. All experiments use the CIC-IDS2017 dataset with four federated clients and focal loss to address class imbalance.

4.1 Experimental Setup

All experiments use the CIC-IDS2017 dataset[15][1]. Alternative datasets such as Bot-IoT and TON_IoT were considered but excluded due to extreme class imbalance. The dataset is split into an 80:20 train/test partition using stratified sampling. Majority-class undersampling is applied to the training data to obtain an approximate 80:20 normal-to-attack ratio, while the test set remains unchanged. We simulate a federated learning environment with four clients following the standard FL paradigm [12]. Each client trains locally while the server aggregates updates using FedAvgM and applies cosine learning-rate decay to stabilize training.

[1]Canadian Institute for Cybersecurity Intrusion Detection System Dataset (CIC-IDS2017): https://www.unb.ca/cic/datasets/ids-2017.html

4.2 Compression Pipeline

The compressed models are produced using a multi-stage compression pipeline. As summarized in Table 2, the final compressed model reduces model size from 0.78 MB to 0.0635 MB (12.28× compression) and lowers inference latency from 1.89 ms to 0.48 ms, corresponding to a 74.5% latency reduction.

First, BatchNorm folding merges BatchNormalization parameters into the preceding Dense layer. The updated weights W_{fold} and bias b_{fold} are computed from $(\gamma, \beta, \mu, \sigma)$ as

$$W_{fold} = \frac{\gamma W}{\sqrt{\sigma^2 + \epsilon}}, \qquad b_{fold} = \beta + \frac{\gamma(b - \mu)}{\sqrt{\sigma^2 + \epsilon}}.$$

Next, a smaller student model is trained using progressive knowledge distillation, where the federated model serves as the teacher. The student network contains 50% of the original parameters. Structured pruning is then applied to remove low-importance neurons from Dense layers. Finally, a quantization-aware fine-tuning step is optionally applied to prepare the model for INT8 deployment.

4.3 Training Stability and Detection Performance

Early experiments on the CIC-IDS2017 dataset revealed that a naive federated training setup with a fixed learning rate leads to highly unstable optimization. Although the global model often achieved high overall accuracy, its recall for the attack class remained low: in a representative run, the model correctly detected only 46.7% of attack flows, indicating that many malicious connections were systematically misclassified as benign. To address this issue, we combine focal loss with a server-coordinated cosine learning-rate schedule. Focal loss down-weights easy benign samples and amplifies gradients from hard attack examples, while cosine decay anneals the global learning rate over T federated communication rounds to avoid late-stage oscillations. With this configuration, our federated baseline improves recall for the attack class from 46.7% to 93.85% on CIC-IDS2017, while maintaining high overall accuracy and eliminating divergence observed in the fixed learning-rate runs.

This stable training regime serves as the foundation for our compression study. Starting from the converged federated model, we apply our compression pipeline to generate 48 configurations that combine FL QAT, knowledge distillation (none, direct, progressive), structured pruning (`none`, `10x5`, `10x2`, `5x10`), and optional post-training quantization. Across these compressed models, most configurations preserve recall close to the 93.85% baseline before compression. In particular, well-performing configurations retain recall above 90% while significantly reducing model size. This indicates that our compression pipeline enables substantial model size reduction while preserving detection performance.

Table 1. Detection performance before and after compression.

Model	Accuracy	F1-score	Recall (Attack)
Baseline Model	93.5%	84.1%	46.7%
Compressed Model	**96.02%**	**89.32%**	**93.85%**

4.4 Detection Performance

Table 1 summarizes detection performance before and after applying the compression pipeline. The baseline federated model exhibits unstable convergence and low recall for the attack class (46.7%). After introducing server-coordinated cosine learning-rate decay, detection performance improves substantially, reaching 96.02% accuracy, 89.32% F1-score, and 93.85% recall for the attack class.

The compressed model is obtained by applying BatchNorm folding, knowledge distillation, structured pruning, and quantization-aware fine-tuning to this stabilized model. Therefore, the performance improvement primarily results from the stabilized training process, while the compression pipeline reduces model size and latency while preserving detection performance.

4.5 Compression and Quantization Analysis

We analyze the interaction between quantization, knowledge distillation, and structured pruning in the federated TinyML compression pipeline. In total, 48 configurations are evaluated, comparing two strategies: training with and without quantization-aware training (QAT) during federated learning. Table 2 summarizes compression configurations and detection performance under moderate and extreme compression levels. Under moderate compression (approximately 38 KB), the configuration without QAT achieves the best performance, reaching 99.16% accuracy and 97.60% F1-score. Enabling QAT during federated training slightly reduces detection performance at this compression level. However, when compression is pushed further to approximately 14 KB models, the trend reverses. In this regime, the configuration using QAT achieves higher accuracy (95.81%) and F1-score (86.66%) compared to the no-QAT configuration. These results suggest that the benefit of QAT depends on the compression regime. While full-precision training performs better under moderate compression, QAT becomes important under extreme compression, where it helps the model adapt to quantization noise and maintain detection performance.

These observations suggest that quantization-aware training (QAT) is not always beneficial during the federated training stage. In our experiments, configurations trained without QAT during FL sometimes achieve higher detection performance under moderate compression levels. However, when models are aggressively compressed, QAT helps stabilize the

Table 2. Detection performance and corresponding compression configurations under moderate and extreme compression.

Compression	Method	FL QAT	Distillation	Pruning	PTQ	Model Size	Accuracy	F1-score
Moderate	No QAT during FL	No	Progressive	10×5	No	37.89 KB	**99.16%**	**97.60%**
Moderate	QAT during FL	Yes	Progressive	10×5	No	37.91 KB	97.91%	94.33%
Extreme	No QAT during FL	No	Progressive	5×10	Yes	14.44 KB	93.54%	77.63%
Extreme	QAT during FL	Yes	Progressive	5×10	Yes	14.44 KB	**95.81%**	**86.66%**

network against quantization noise and improves performance under aggressive compression.

This behavior is consistent with prior studies showing that QAT may introduce optimization instability due to oscillations between quantization grid points during training [14]. Moreover, several analyses indicate that post-training quantization (PTQ) can achieve near floating-point performance at 8-bit precision without requiring QAT in many practical scenarios [13]. These findings highlight that the benefit of QAT depends strongly on the compression regime and training configuration.

5 Future Work

Future work will focus on three directions.

Dynamic Communication and Adaptive Aggregation: We plan to reduce communication overhead by exploring selective client participation, where only devices with significant updates transmit model parameters. We will also investigate asynchronous federated aggregation to handle heterogeneous network conditions across IoT nodes.

Adversarial Robustness and Training Strategies: While this work focuses on improving training stability and compression efficiency, future work will extend the framework to include adversarial robustness. In particular, we plan to incorporate adversarial training techniques within the federated learning pipeline and evaluate robustness under stronger attacks such as Projected Gradient Descent (PGD). This will allow us to study the trade-off between model compression, detection performance, and robustness in resource-constrained environments.

Real-World Deployment on Microcontrollers: We will further validate the proposed pipeline through end-to-end deployment on microcontroller-class devices such as ESP32 using TensorFlow Lite for Microcontrollers. This includes measuring on-device inference latency, memory usage, and energy consumption to ensure that the compressed models operate reliably under real-world hardware constraints.

6 Conclusion

This work investigates the feasibility of deploying federated intrusion detection models on resource-constrained IoT devices by integrating TinyML-oriented compression techniques. Our results show that stabilizing federated training plays a critical role in improving detection performance, increasing Attack Recall from 46.7% to 93.85% using server-coordinated cosine learning-rate scheduling. The proposed compression pipeline further enables efficient edge deployment, achieving a 12.28× reduction in model size and a 74.5% reduction in inference latency on ESP32-class hardware while preserving detection accuracy.

Our analysis also reveals that the effectiveness of quantization-aware training depends on the compression regime: full-precision training performs better under moderate compression, whereas QAT becomes important when models are aggressively compressed. These findings provide practical guidance for designing lightweight and privacy-preserving intrusion detection systems suitable for federated TinyML deployment in IoT environments.

References

[1] Luigi Atzori, Antonio Iera, and Giacomo Morabito. 2010. The Internet of Things: A Survey. Computer Networks 54, 15 (2010), 2787–2805.

[2] Colby Banbury et al. 2020. TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. arXiv preprint arXiv:2003.05403 (2020).

[3] Ian Goodfellow et al. 2015. Explaining and Harnessing Adversarial Examples. In ICLR.

[4] Song Han et al. 2016. Deep Compression: Compressing Deep Neural Networks. ICLR (2016).

[5] Geoffrey Hinton et al. 2015. Distilling the Knowledge in a Neural Network. arXiv preprint arXiv:1503.02531 (2015).

[6] Benoit Jacob et al. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In CVPR.

[7] Ahmad Javaid et al. 2016. A Deep Learning Approach for Network Intrusion Detection System. In EAI Bio-Inspired ICT.

[8] Peter Kairouz et al. 2021. Advances and Open Problems in Federated Learning. Foundations and Trends in Machine Learning (2021).

[9] Jakub Konečný et al. 2016. Federated Learning: Strategies for Improving Communication Efficiency. arXiv preprint arXiv:1610.05492 (2016).

[10] Tian Li et al. 2020. Federated Optimization in Heterogeneous Networks. In MLSys.

[11] Ilya Loshchilov and Frank Hutter. 2017. SGDR: Stochastic Gradient Descent with Warm Restarts. In ICLR.

[12] H. Brendan McMahan et al. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In AISTATS.

[13] Markus Nagel et al. 2021. A White Paper on Neural Network Quantization. arXiv preprint arXiv:2106.08295 (2021).

[14] Markus Nagel et al. 2022. Overcoming Oscillations in Quantization-Aware Training. In ICML.

[15] Iman Sharafaldin et al. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In ICISSP.